The Principles of Diffusion Models

1.2 Prominent Deep Generative Models

渡邉 有志

October 14, 2026

主要な深層生成モデリング

生成モデリングにおける中心的な課題

高次元データの豊かで複雑な構造を捉えられる
表現力の高い確率モデルの学習

計算の扱いやすさ、表現力、学習効率の間で異なるトレードオフ
⇒代表的な戦略を紹介し、計算グラフの比較とともに説明する

1.2 Prominent Deep Generative Models

定義:EBMs(Energy-based Models)

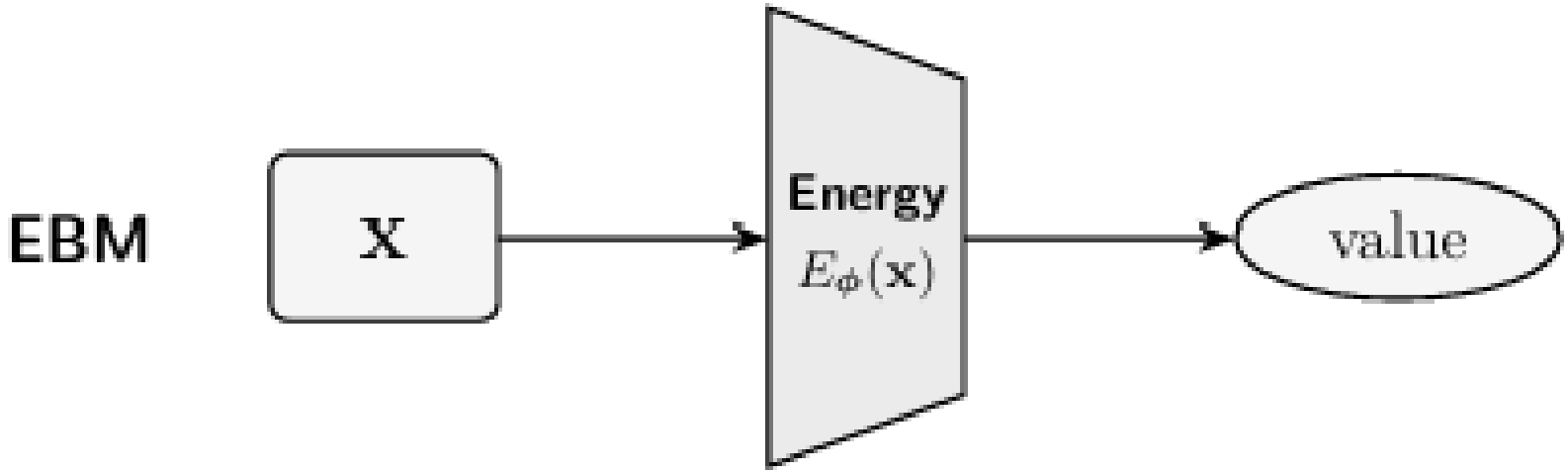
EBMs: エネルギー関数 $E_\phi(x)$ を通して確率分布を定義するモデル

$$p_\phi(x) := \frac{1}{Z(\phi)} \exp(-E_\phi(x))$$

ここで $Z(\phi) = \int \exp(-E_\phi(x))dx$ は分配関数、これを代入すると

$$p_\phi(x) = \frac{\exp(-E_\phi(x))}{\int \exp(-E_\phi(x))dx}$$

- $exp(-E_\phi(x)) > 0$
- 分配関数で割ることで全体の積分が 1 になる
- もし x が離散変数なら $-E_\phi(x)$ に対してソフトマックスを適用したものに近い
  ⇒連続変数の出力は確率密度なので 1 を越えてもいい。
  必要なのは全体の積分が 1 になること



エネルギー関数: 高次元データ x に対して 1 つの実数 $E_\phi(x)$ を割り当てている $(E_\phi(x) : R^D \to R)$
$E_\phi(x)$ は正規化定数が有限の範囲内で柔軟に設計可能
実数全体で $E_\phi(x) = 0 \implies Z = \int_{-\infty}^{\infty} 1dx = \infty$ より正規化できない

EBM を通してエネルギー→確率密度の変換を行っている
EBM の学習ではデータの対数尤度を最大化する
しかし、分配関数が計算困難で対処が必要
⇒次の章で拡散モデルについて解説

AutoRegressive Models

定義:AutoRegressive Models(自己回帰モデル)

AR モデル: 各データ点が前のデータ点に依存して生成されるモデル

これは確率の連鎖率を用いることで、どんな同時分布も一連の条件付確率に分解が可能[1]
変数 $x = (x_1, x_2, \ldots, x_d)$ に対して、
$p_{data}(x) = \prod_{i=1}^{d} p(x_i|x_{<i}), x_{<i} = x_1, \ldots, x_{i-1}$ と表せる
(モデル化の仮定ではなく変数について順不同の厳密な恒等式)

問題は $p_{data}(x_i \mid \boldsymbol{x}_{<i})$ が未知であること
⇒ AR モデルは同じ因数分解の形を持つモデルをパラメータ化
$\log p_\phi(x) = \sum_{i=1}^{D} \log p_\phi(x_i \mid \boldsymbol{x}_{<i})$

[1] chain rule:next slide

AutoReguression Models

NN の出力 (e.x. 語彙生成) ⇒条件付分布の近似

モデルに系列トークンを入力し、次のトークンの確率分布を出力

入力に対し次トークンの候補を数値化し、softmax で確率に変換

$$p_\phi(x_i = k \mid \boldsymbol{x}_{<i}) = \frac{\exp(a_k)}{\sum_{k' \in \mathcal{V}} \exp(a_{k'})}$$

- 確率は非負で候補全体の総和が 1
- a_k は前の要素 $x_{<i}$ とパラメータ ϕ によって変わる

連続値の場合、正規化分布の平均と標準偏差を出力し、
$p_{/phi}(x_i \mid x_{<i}) = \mathcal{N}(\mu_\phi(x_{<i}), \sigma_\phi(x_{<i})^2)$

- 各位置に別々のネットワークが必要とは限らない
- 全ての位置で同じパラメータを使うことも可能
- 入力される前の要素に応じて異なる条件付分布を出力可能

AutoReguression Models

なぜ積をとるだけで全体が正規化されるのか

三つの離散変数の場合

$p_{data}(x_1, x_2, x_3) = p_{data}(x_1 \mid x_2, x_3) p_{data}(x_2 \mid x_3) p_{data}(x_3)$

全ての値について足すと

$\sum_{x1} p_\phi(x_1) \sum_{x2} p_\phi(x_2 \mid x_1) \sum_{x3} p_\phi(x_3 \mid x_1, x_2) = 1$

最後の変数から順に足していくと条件付分布の総和が 1 になり、全体も 1 になる。連続なら和を積分に置き換える

EBM ではデータ空間全体の正規か定数が問題

⇒ AR モデルでは全体正規化が保証される

学習について

尤度に基づく学習が容易になる

データサンプル $x^{(1)}, \ldots, x^{(N)}$ に対応する対数尤度は
$\log p_\phi(x) = \sum_{i=1}^{D} \log p_\phi(x_i \mid x_{<i})$ のように分解される

- 最尤推定によって直接学習可能
  ⇒これは負の対数尤度を最小かすることと同値

- 観測済みの先行変数 $x_{<i}$ が利用できる
  ⇒条件付き予測の並列計算が可能

生成について

生成は逐次的に行われる

生成では正解の列がないためまず一つ目を生成する

$x_1 \sim p_\phi(x_1)$

その結果を使って

$x_2 \sim p_\phi(x_2 \mid x_1)$

これを繰り返していくことで生成が可能

⇒次の分布を決める条件が前の生成結果に依存する

ここでの生成は最も確率が高い候補を選ぶということではなく

分布からランダムにサンプリングする

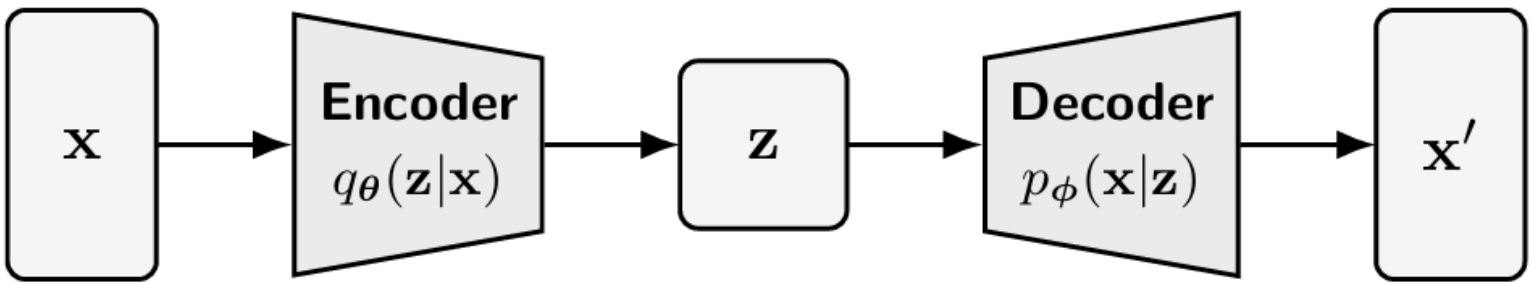
⇒同じ条件で異なるデータを生成可能

VAE:variational Autoencoder

VAE(変分オートエンコーダ)

VAE: データに隠れた構造である潜在変数 z を学習するモデル
(オートエンコーダの拡張)
xとz 間の写像を直接学習する代わりに確率的な見方を採用する
エンコーダとデコーダの両方を学習する

- エンコーダ $q_\theta(z \mid x)$:
  ⇒データが与えられたときの潜在変数の未知の分布を近似
- デコーダ $p_\phi(x \mid z)$:
  ⇒潜在変数からデータを再構成する



AE(AutoEncoder)

AE と VAE の違い

- 学習するもの
  AE: 入力を復元する写像
  VAE: 潜在変数を介してデータを生成する確率モデル

ELBO(Evidence Lower Bound)

ELBO(Evidence Lower Bound)

$$L_{ELBO}(\theta, \phi; x) = \mathbb{E}_{z \sim q_\theta(z|x)}[\log p_\phi(x \mid z)] - D_{KL}(q_\theta(z \mid x) \,||\, p(z))$$

真の対数尤度の代わりに計算可能なエビデンス下界を最大化する

第一項はデータの正確な再構成を促す項
第二項は潜在変数の分布 $p_{perior}(z)$ を単純な事前分布に近づけ、正則化する項

【VAE について】
ニューラルネットワークと潜在変数モデルを組み合わせる方法で尤度に基づくアプローチとして最も多く使われている一つの手法

生成サンプルの鮮明さが不十分、学習上の問題などの実用上の課題がありつつも拡散モデルなどの発展の基礎となっている

ELBO の導出

対数尤度に潜在変数 z における任意の分布 $q(z)$ を用いて

$$
\begin{aligned}
\log p_\theta(\mathbf{x}) &= \log p_\theta(\mathbf{z} \mid \mathbf{x}) \int q(z) d\mathbf{z} \\
&= \int q(z) \log p_\theta(\mathbf{x}) d\mathbf{z} \\
&= \int q(z) \log \frac{q(z) p_\theta(\mathbf{z}, x) p_\theta(\mathbf{x})}{q(z) p_\theta(\mathbf{z} \mid \mathbf{x})} d\mathbf{z} \\
&= \int q(z) \log \frac{p_\theta(\mathbf{z}, x) p_\theta(\mathbf{x})}{q(z)} d\mathbf{z} + \int q(z) \log \frac{q(z)}{p_\theta(\mathbf{z} \mid \mathbf{x})} d\mathbf{z} \\
&= L(\theta, \phi; x) + D_{KL}(q(z) \,||\, p_\theta(\mathbf{z} \mid \mathbf{x}))
\end{aligned}
$$

ただし $L(\theta, \phi; x) = \int q(z) \log \frac{p_\theta(\mathbf{z}, x)}{q(z)} d\mathbf{z}$
第一項は ELBO、第二項は KL ダイバージェンス
第二項は 0 以上なので ELBO は必ず周辺尤度の下界を与える
$log p_\theta(\mathbf{x}) \geq L(\theta, \phi; x)$

Nomalization Flows

従来の正規化フロー：

正規化フロー (NF) とニューラル常微分方程式 (NODE) に基づく連続時間のモデル
⇒可逆の変換を用いて、単純な潜在変数と複雑なデータの分布を結び付ける全単射を学習

$$\log p_\phi(x) = \log p(f_\phi(x)) + \log \left| \det \frac{\partial f_\phi(x)}{\partial x} \right|, z = f_\phi(x), x = f_\phi^{-1}(z)$$

概念的には美しいが、実装上の制約が多い

- 全単射を保証するためネットワーク構造に厳しい制約
- 常微分方程式を解く計算負荷が高く学習効率低下
- 可逆で、ヤコビアンの行列式も効率よく計算できる変換

解釈

目的：実際のデータに近い分布を学習し、新しいデータを生成

分布から得たサンプル (z) は潜在変数であり高次元のベクトル
潜在変数を変換して実際のデータに近い形にする
※ここで、生成された x の確率密度が計算できるとは限らない
⇒可逆な変換によって潜在変数 z の確率密度を使えるようにする
$z = f_\phi(x), x = f_\phi^{-1}(z)$

ただし、変換によって空間が伸び、単位体積当たりの確率密度が変わるので対数変換の公式を用いて対策する
$p_\phi(x) = p(f_\phi(x)) \left|\det \frac{\partial f_\phi(x)}{\partial x}\right|,$
対数をとって $\log p_\phi(x) = \log p(f_\phi(x)) + \log \left|\det \frac{\partial f_\phi(x)}{\partial x}\right|$

学習データを大きくするように、パラメータ ϕ を学習可能になる
⇒「変数変換の公式によって最尤推定が可能になる」

GANs:Generative Advasarial Networks

GANs:Generative Advasarial Networks

GANs: 生成器 G と判別機 D が互いに競い合う二つの NN

- 生成器 G: ランダムなノイズ $z \sim P_{prior}$ から
  現実的なサンプル $G_\phi(z)$ を作ることを目指す
- 判別機 D: 実サンプル x か生成サンプル $G_\phi(z)$ かを判別する

GANs の目的関数:

$$\min_{G_\phi} \max_{D_\zeta} \left\{ \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [\log D_\zeta(x)]}_{\text{実サンプル}} + \underbrace{\mathbb{E}_{z \sim p_{\text{prior}}} [\log (1 - D_\zeta(G_\phi(z)))]}_{\text{生成サンプル}} \right\}.$$

- 明示的な確率密度関数を定義しない⇒尤度の推定を回避
- 正規化定数を計算せず、よく模倣するサンプルの生成に重点

ダイバージェンスの観点から見ると

D_ζ が真の分布 p_{data} と生成器の分布 p_{G_ϕ} の間の隔たりを暗黙的に測る
※ p_{G_ϕ} は、ノイズ $z \sim p_{\text{prior}}$ から得られる生成サンプル $G_\phi(z)$ の分布
生成器 G_ϕ を固定したとき、最適な識別器は

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{G_\phi}(x)}$$

この最適な最適な識別機を用いると生成器の最小化問題は

$$\min_{G_\phi} \left[2D_{\text{JS}}(p_{\text{data}} \| p_{G_\phi}) - \log 4 \right]$$

ここで D_{JS} は次のように定義される

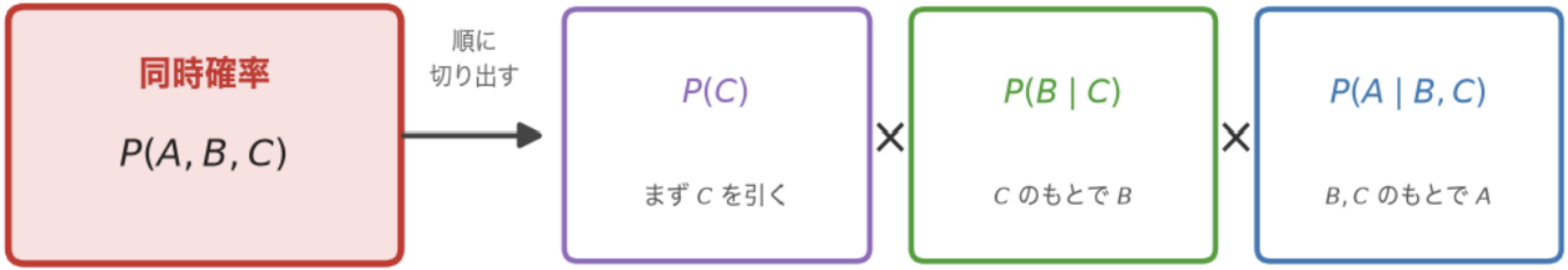
$$D_{\text{JS}}(p\|q) := \frac{1}{2} D_{\text{KL}} \left(p \middle\| \frac{p+q}{2} \right) + \frac{1}{2} D_{\text{KL}} \left(q \middle\| \frac{p+q}{2} \right).$$

GAN が暗黙的に $D_{\text{JS}}(p_{\text{data}} \| p_{G_\phi})$ を最小化していることを示している

chain rule

chain rule: 確率の連鎖律 (確率の乗法定理)

複数の確率変数の同時確立を条件付確率の積として表す公式
イメージ：



条件付確率 $P(A \mid B) = \frac{P(A,B)}{P(B)}$
両辺に $P(B)$ をかけると $P(A, B) = P(A \mid B)P(B)$

3 変数の場合

$P(A, B, C) = P(A \mid B, C)P(B \mid C)P(C)$
ここで $P(B, C)$ は 2 変数の同時確立なのでもう一度乗法定理
$P(B, C) = P(B \mid C)P(C)$ より
$P(A, B, C) = P(A \mid B, C)P(B \mid C)P(C)$

chain rule の一般化

chain rule の一般化

n 個の確率変数 $A_1, A_2, ..., A_n$ の同時確率からまず A_1 を切り出すと
$P(A_1, A_2, ..., A_n) = P(A_1 \mid A_2, ..., A_n)P(A_2, A_3, ..., A_n)$
ここで $P(A_2, A_3, ..., A_n)$ は n-1 個の確率変数の同時確率なので
もう一度乗法定理によって
$P(A_1, A_2, ..., A_n) = P(A_1 \mid A_2, ..., A_n)...P(A_{n-1} \mid A_n)P(A_n)$
したがって

$$P(A_1, A_2, ..., A_n) = \prod_{i=1}^{n} P(A_i \mid A_{i+1}, ..., A_n)$$

※最初の項は $P(A_1)$ になる

どの変数にも対称に使え、別の順序でも同じ同時確率が得られる
⇒問題ごとに計算しやすい順序を選べる